

Loss و logits، موتور آموزش PyTorch، و ابزارهای generalization

نقشه‌ی زمانی ۱۸۰ دقیقه

منبع	نوع فعالیت غالب	تمرکز	بلوک	زمان
Session 1 · بلوک ۰	اجرا و مشاهده (کد آماده)	بارگذاری CIFAR-10، شناخت داده	راه‌اندازی	۱۰-۰
cross_entropy.ipynb	کدنویسی از صفر + تصمیم	logits، BCE، Loss در برابر CE	بلوک A	۴۰-۱۰
Session 1	کدنویسی کامل از صفر + سؤال مفهومی	موتور آموزش کامل: CNN، loss، optimizer، حلقه	بلوک B	۹۰-۴۰
—	—	—	استراحت	۱۰۰-۹۰
Session 2	کدنویسی کامل از صفر + آزمایش	جعبه‌ابزار Generalization روی همان مدل	بلوک C	۱۵۵-۱۰۰
ترکیبی	دیباگ + بازنویسی کامل از صفر	چالش یکپارچه‌ی پایانی + جمع‌بندی	بلوک D	۱۸۰-۱۵۵

بلوک ۰ – راه‌اندازی (۱۰-۰ دقیقه)

این کد را دقیقاً مثل فایل مرجع اجرا کنید؛ این پایه‌ی کل جلسه است – بعداً تغییرش نمی‌دهیم:

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", DEVICE)

image_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
])

train_full = datasets.CIFAR10(root="data", train=True, download=True, transform=image_transform)
test_full = datasets.CIFAR10(root="data", train=False, download=True, transform=image_transform)

TRAIN_LIMIT = 4096
TEST_LIMIT = 1000
train_dataset = Subset(train_full, range(TRAIN_LIMIT))
test_dataset = Subset(test_full, range(TEST_LIMIT))
```

```

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=0)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False, num_workers=0)

class_names = train_full.classes
print("کلاس‌ها:", class_names)

images, labels = next(iter(train_loader))
print("شکل تماویر:", images.shape)
print("شکل لیبل‌ها:", labels.shape)

```

سؤال ۱۰.۱ (تثبیتی) – ۳ دقیقه

طبق منطق DataLoader در Session 1 (بخش «چرا mini-batch؟»)، TRAIN_LIMIT و TEST_LIMIT چرا اینجا محدود شده‌اند؟ اگر این محدودیت را در یک اجرای واقعی (نه کلاسی) حذف کنیم، دقیقاً چه چیزی به نفع چه چیزی قربانی می‌شود؟

بلوک Loss، Logits – A و تصمیم درست ۳۰ دقیقه

هدف این بلوک: برای یک مسئله‌ی واقعی (CIFAR-10) تصمیم درست را بگیرید، سپس با کدی که خودتان می‌نویسید ثابت کنید تصمیم‌تان درست بوده.

A1 – تشخیص نوع مسئله ۵ دقیقه

CIFAR-10 هر تصویر را دقیقاً در یکی از ۱۰ کلاس class_names قرار می‌دهد.

سؤال

طبق جدول مقایسه‌ی cross_entropy.ipynb (بخش «BCE در برابر Cross-Entropy»)، کدام Loss باید استفاده شود – BCEWithLogitsLoss یا CrossEntropyLoss؟ دقیقاً با اشاره به همان جدول توضیح دهید: مسئله‌ی CIFAR-10 در کدام ردیف جدول (رقابت انحصاری بین کلاس‌ها، یا چند تصمیم بله/خیر مستقل) قرار می‌گیرد، و چرا همین تفاوت باعث می‌شود اینجا انتخاب نادرستی باشد؟

A2 – کدنویسی از صفر: مدل ابتدایی و ردیابی logits ۸ دقیقه

بدون کپی، خودتان از صفر بنویسید.

1. یک nn.Sequential به نام toy_model بسازید که یک تصویر $32 \times 32 \times 3$ را Flatten کند و با یک لایه‌ی nn.Linear مستقیماً به تعداد کلاس‌ها (len(class_names)) نگاشت دهد. مدل را به DEVICE منتقل کنید.
2. یک batch واقعی از train_loader بگیرید، images و labels را به DEVICE منتقل کنید، و logits = toy_model(images) را محاسبه کنید.
3. شکل logits را چاپ کنید.

سوالات مفهومی A2

- چرا شکل logits باید [10, 64] باشد؟ عدد ۶۴ از کجا آمد و عدد ۱۰ از کجا؟
- اگر batch_size را در بلوک ۰ به ۳۲ تغییر دهیم، کدام یک از این دو عدد تغییر می‌کند و کدام ثابت می‌ماند؟ چرا؟

A3 – کدنویسی از صفر: اثبات فرمول Cross-Entropy با دست ۱۰ دقیقه

cross_entropy.ipynb فرمول را این طور نوشته: $L = -\sum y_i \log(p_i)$ ، که برای یک لیبل صحیح فقط به $\log(p_{\text{correct}})$ ساده می شود. بدون نگاه به کد فایل، فقط با تکیه بر همین فرمول، کد زیر را خودتان بنویسید:

1. `critterion = nn.CrossEntropyLoss()` بسازید و `loss_pytorch = critterion(logits, labels)` را با `logits` و `labels` خودتان از A2 محاسبه کنید.
2. به صورت دستی، برای همان `probabilities = torch.softmax(logits, dim=1)` را بگیرید، سپس با اندیس گذاری `probabilities[torch.arange(len(labels)), labels]` احتمال کلاس صحیح هر نمونه را استخراج کنید.
3. میانگین $\log$ - این احتمال ها را روی کل batch محاسبه کنید و آن را `loss_manual` بنامید.
4. چاپ کنید و مقایسه کنید: آیا `loss_pytorch` و `loss_manual` تا چند رقم اعشار برابرند؟

سؤالات مفهومی A3

- مدل شما هنوز هیچ آموزشی ندیده است. طبق منطق فایل) بخش Cross-Entropy «برای چند کلاس»، وقتی مدلی کاملاً تصادفی به هر ۱۰ کلاس تقریباً احتمال یکسان بدهد، فرمول $L = -\log(p_{\text{correct}})$ چه مقداری پیش بینی می کند؟ ابتدا خودتان با جایگذاری $p_{\text{correct}} \approx 1/10$ در فرمول این عدد را روی کاغذ حساب کنید، سپس با کد زیر تأیید کنید:

```
import math
print("پیش بینی شما: ...") # عدد محاسبه شده ی خودتان را اینجا بگذارید
print(-math.log(1 / len(class_names)))
print("loss واقعی مدل: ", loss_pytorch.item())
```

- آیا این سه عدد به هم نزدیک اند؟
- اگر مدل روی یک نمونه ی خاص خیلی مطمئن ولی اشتباه پیش بینی کند (مثلاً $p_{\text{correct}}=0.01$)، `loss_manual` برای آن یک نمونه با فرمول $-\log(p_{\text{correct}})$ چه عددی می شود؟ این را با یک نمونه ی دیگر که $p_{\text{correct}}=0.9$ دارد مقایسه کنید – این تفاوت چه چیزی درباره ی رفتار Cross-Entropy نشان می دهد؟

A4 – چرا هرگز softmax را دستی قبل از Loss نمی زنیم ۷ دقیقه

- وقتی عدد داخل e^z خیلی بزرگ باشد، کامپیوتر نمی تواند آن را به درستی نگه دارد و نتیجه به `inf` (بی نهایت) یا `nan` (نامعتبر) میل می کند – به این پدیده «ناپایداری عددی» می گویند. cross_entropy.ipynb (بخش «چرا softmax و cross-entropy»)) دقیقاً همین مشکل را با عددهایی مثل ۱۰۰۰, ۹۹۹, ۹۹۸ نشان می دهد. بدون نگاه به آن کد، خودتان از صفر بنویسید:
1. یک تانسور `logits` با چند عدد نسبتاً بزرگ و نزدیک به هم بسازید (مثلاً چند عدد ساده را در ۲۰ یا ۵۰ ضرب کنید تا مصنوعاً بزرگ شوند) و یک `target` مربوط به آن.
 2. فرمول خام $p_i = e^{z_i} / \sum_j e^{z_j}$ را دقیقاً همین طور، بدون هیچ ترفند `safety`، پیاده سازی کنید – این نسخه را عمداً نسخه ی بدون محافظت می نامیم چون منتظریم مشکل رخ بدهد.
 3. $\log$ - احتمال کلاس صحیح را از این نسخه حساب کنید و چاپ کنید – دنبال `nan` یا `inf` بگردید.
 4. همان `logits` و `target` را مستقیماً به `nn.CrossEntropyLoss()` بدهید و نتیجه را چاپ کنید.

سؤالات مفهومی A4

- نسخه ی دستی شما چه مقداری تولید می کند – یک عدد معتبر، `nan` یا `inf`؟ چرا `nn.CrossEntropyLoss` با همان ورودی مشکلی ندارد؟

- در پاسخ خود به «log-sum-exp trick» که فایل نام می‌برد اشاره کنید. این آزمایش چرا دلیل عملی دومی است (جدای از مشتق‌پذیری argmax) که همیشه logits خام را به criterion می‌دهیم، نه خروجی softmax یا argmax را؟

بلوک B – موتور آموزش کامل، از صفر ۵۰ دقیقه

در این بلوک هیچ کد آماده‌ای دریافت نمی‌کنید (به‌جز بخش رفع باگ در پایان). فقط شرح معماری و رفتار مورد نظر داده می‌شود؛ مدل، optimizer، loss، توابع آموزش/ارزیابی و حلقه‌ی آموزش را کاملاً خودتان می‌نویسید.

B1 – طراحی روی کاغذ، قبل از کدنویسی ۵ دقیقه

معماری هدف (دقیقاً همان SimpleCNN که در Session 1 تدریس شد) این‌طور توصیف می‌شود:

- یک بخش $\text{features: Conv2d}(3 \rightarrow 16, \text{kernel_size}=3, \text{padding}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(\text{kernel_size}=2)$
 $\text{Conv2d}(16 \rightarrow 32, \text{kernel_size}=3, \text{padding}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(\text{kernel_size}=2)$
- یک بخش classifier: Flatten $\rightarrow$ Linear (به تعداد کلاس‌ها)

سؤال (بدون اجرای کد)

ورودی CIFAR-10 پس از image_transform شکل [B, 3, 32, 32] دارد. با محاسبه‌ی دستی (نه اجرای کد)، شکل تنسور را بعد از هر لایه‌ی features بنویسید. لایه‌ی Linear باید دقیقاً با چه عددی ورودی بگیرد؟ این عدد را بعد از نوشتن کد در B2 با کد واقعی هم تأیید کنید.

B2 – کدنویسی کامل از صفر: مدل، Loss، Optimizer، حلقه‌ی آموزش ۲۵ دقیقه

بدون کپی از فایل‌ها، از صفر بنویسید:

1. کلاس SimpleCNN(nn.Module) را دقیقاً طبق معماری B1 بسازید؛ سازنده‌اش num_classes می‌گیرد. متد forward باید features را روی ورودی اعمال کند و خروجی را به classifier بدهد.
2. یک نمونه از این مدل با $\text{num_classes}=\text{len}(\text{class_names})$ بسازید و به DEVICE منتقل کنید. سپس Loss criterion مناسب طبق بلوک A و optimizer (Adam با $\text{lr}=1\text{e-}3$) را بسازید.
3. تابعی با امضای دقیق $\text{train_one_epoch}(\text{model}, \text{loader}, \text{criterion}, \text{optimizer}, \text{device})$ بنویسید که یک epoch کامل را روی loader طی کند و میانگین loss و دقت (accuracy) آن epoch را برگرداند. خودتان تصمیم بگیرید داخل این تابع چه کارهایی لازم است (جابه‌جایی به device، پاک کردن گرادینان قدیمی، محاسبه‌ی logits، محاسبه‌ی loss، پس‌انتشار، به‌روزرسانی وزن‌ها، و شمارش صحیح نمونه‌های درست).
4. تابعی با امضای دقیق $\text{evaluate}(\text{model}, \text{loader}, \text{criterion}, \text{device})$ بنویسید که همان معیارها را روی داده‌ی ارزیابی حساب می‌کند – اما بدون یادگیری. خودتان تصمیم بگیرید این تابع چه تفاوت‌هایی با train_one_epoch باید داشته باشد.
5. با این دو تابع، مدل را ۲ epoch روی $\text{train_loader}/\text{test_loader}$ (بلوک ۰) آموزش دهید و در پایان هر epoch دقت train و test را چاپ کنید. نتیجه را history نگه دارید تا در بلوک C دوباره به کار برود.

سؤالات مفهومی B2 (بعد از نوشتن کد)

- چرا model، images و labels باید همگی روی یک device باشند؟ اگر یکی جا بماند، PyTorch دقیقاً چه خطایی نشان می‌دهد؟
- چرا در هر batch باید گرادینان‌های قبلی را پاک کنیم؟ فایل Session 1 درباره‌ی رفتار پیش‌فرض PyTorch نسبت به گرادینان‌ها چه می‌گوید؟ اگر این پاک‌سازی را حذف کنید چه اتفاقی برای گرادینان‌های بعدی می‌افتد؟
- در evaluate خودتان چه تصمیمی درباره‌ی $\text{model.eval}()$ و $\text{torch.no_grad}()$ گرفتید؟ حالا با استدلال فایل Session 1 مقایسه کنید – این دو مورد چرا لازم‌اند و چه اتفاقی می‌افتاد اگر آن‌ها را نمی‌گذاشتید؟

- چرا criterion باید logits خام بگیرد، نه خروجی softmax؟ (به بلوک A و cross_entropy.ipynb ارجاع دهید).
- آیا دقت شما از حدس تصادفی $10/1 = 0.1$ بهتر است؟ اگر نه، طبق چک‌لیست دیباگ فایل «Common bug» و «Pair debugging») چه سه چیز را اول چک می‌کنید؟

B3 – Instrumentation: مشاهده‌ی یک گرادیان واقعی ۵ دقیقه

این بار هم کد آماده‌ای نیست – کل گام آموزش را خودتان می‌نویسید:

1. وزن اولین لایه‌ی کانولوشن مدل (`model.features[0].weight`) را قبل از هر تغییری، با `detach().clone()` ذخیره کنید.
2. خودتان یک گام کامل آموزش روی یک batch از `train_loader` بنویسید (همان مرحله‌ی که در `train_one_epoch` به کار بردید، اما فقط برای یک batch).
3. همان وزن را بعد از این یک گام دوباره بگیرید و میانگین قدر مطلق تفاوت دو نسخه را چاپ کنید.

سؤالات مفهومی B3

- اگر خط `optimizer.step()` را عمداً حذف کنید، عدد چاپ‌شده چه می‌شود؟
- چرا وجود یک گرادیان غیرصفر (بعد از backward) به‌تنهایی تضمین نمی‌کند که به‌روزرسانی وزن بزرگ باشد؟ نرخ یادگیری (learning rate) در این‌جا چه نقشی دارد؟

B4 – چالش گروهی: رفع باگ ۱۰ دقیقه

کد زیر ۴ باگ دارد که دقیقاً همان الگوهایی هستند که فایل به‌عنوان «اشتباهات رایج» (بخش Pair debugging challenge) فهرست می‌کند:

```
buggy_model = SimpleCNN(num_classes=len(class_names)) # بدون .to(DEVICE)
buggy_optimizer = torch.optim.Adam(buggy_model.parameters(), lr=1e-3)

for images, labels in train_loader:
    images = images.to(DEVICE)
    predictions = torch.softmax(buggy_model(images), dim=1)
    loss = nn.CrossEntropyLoss()(predictions, labels)
    loss.backward()
    buggy_optimizer.step()
```

باید هر ۴ مشکل را پیدا کنید، هرکدام را در یک خط توضیح دهید، و نسخه‌ی اصلاح‌شده را کامل – با استفاده از توابع خودتان از B2 – بنویسید.

بلوک C – جعبه‌ابزار Generalization روی همان مدل ۵۵ دقیقه

C1 – Preprocessing یا Augmentation؟ ۵ دقیقه

`transforms.Normalize` و `transforms.RandomHorizontalFlip` را با هم مقایسه کنید – کدام باید روی هر دو `train/test` اعمال شود و کدام فقط روی `train`؟ چرا؟ (طبق بخش «Preprocessing versus augmentation» فایل Session 2)

C2 – کدنویسی از صفر: پایپ‌لاین augmentation با استدلال ۱۰ دقیقه

بدون کد آماده، خودتان یک `transforms.Compose` به نام `train_augmentation` بنویسید که:

- حداقل سه transform تصادفی شامل کند – مثلاً RandomHorizontalFlip, RandomCrop, ColorJitter – و پارامترهای هرکدام (احتمال، میزان padding، شدت تغییر رنگ و ...) را خودتان انتخاب کنید.
- در پایان، دقیقاً مثل بلوک ۰، ToTensor و همان Normalize را اعمال کند تا خروجی با بقیه‌ی پایپ‌لاین سازگار بماند.
- سپس با همین train_augmentation یک نسخه‌ی augmented از train_full بسازید، آن را به همان TRAIN_LIMIT محدود کنید، و یک augmented_loader با batch_size=64 و shuffle=True بسازید.

سؤالات مفهومی C2

- class_names را چاپ کنید. طبق چک‌لیست سه‌بخشی فایل (حفظ لیبل، واقع‌گرایی، حفظ اطلاعات)، آیا RandomHorizontalFlip برای همه‌ی این ۱۰ کلاس منطقی است؟ آیا کلاسی در CIFAR-10 هست که فلیپ افقی برایش بی‌معنی یا مشکل‌ساز باشد؟
- برای هر پارامتری که در ColorJitter انتخاب کردید (مثلاً brightness=0.2)، یک جمله بنویسید که چرا آن مقدار را زیادت‌ر یا کمتر انتخاب نکردید.

C3 – کدنویسی از صفر: RegularizedCNN و آزمایش کنترل‌شده ۳۰ دقیقه

فقط شرح معماری داده می‌شود، نه کد. خودتان کلاس را از صفر بنویسید:

- سازنده دو آرگومان قابل‌تنظیم می‌گیرد: dropout_p (پیش‌فرض ۰.۰) و pooling (پیش‌فرض "max"، مقدار دیگر "average").
- features: یک ReLU + Conv2d(3→16, kernel=3, padding=1) + یک لایه‌ی pooling که بر اساس آرگومان pooling انتخاب می‌شود (MaxPool2d یا AvgPool2d با kernel=2)، سپس یک Conv2d(16→32, kernel=3, padding=1) + ReLU + MaxPool2d(kernel=2) – این لایه‌ی دوم همیشه Max است.
- classifier: Flatten، سپس Dropout با احتمال dropout_p، سپس یک لایه‌ی Linear که به تعداد کلاس‌ها نگاشت می‌دهد.

سپس تابعی به نام run_experiment بنویسید که یک آزمایش کامل را انجام می‌دهد:

1. امضا: run_experiment(name, loader, dropout_p=0.0, pooling="max", epochs=2)
 2. داخل تابع، یک RegularizedCNN جدید با آرگومان‌های داده‌شده بسازید، به DEVICE منتقل کنید، و criterion/optimizer مناسب (مثل بلوک B) تعریف کنید.
 3. با استفاده مجدد از train_one_epoch و evaluate خودتان از بلوک B (آن‌ها را دوباره بنویسید – فقط صدا بزنید)، حلقه‌ی epochs را بنویسید و train/test accuracy هر epoch را در یک history جمع کنید و چاپ کنید.
 4. تابع باید history را برگرداند.
- در نهایت این سه آزمایش را با کد خودتان اجرا کنید و در یک دیکشنری results نگه دارید: baseline (train_loader، augmentation، dropout (train_loader، dropout_p=0.3)، dropout_p=0.0 از augmented_loader)، C2، C3.

سؤالات مفهومی C3

- آیا نتیجه‌ی results["baseline"] با history بلوک B2 قابل مقایسه است؟ چه چیزی بین این دو مدل متفاوت است – معماری SimpleCNN در برابر RegularizedCNN با dropout_p=0؟
- چرا در run_experiment توانستید train_one_epoch/evaluate را دوباره صدا بزنید بدون این‌که چیزی در آن‌ها را عوض کنید؟ این چه چیزی درباره‌ی طراحی خوب یک «موتور آموزش» نشان می‌دهد؟

C4 – کدنویسی از صفر: رسم و خواندن منحنی‌ها ۱۰ دقیقه

خودتان کد رسم را بنویسید: با یک حلقه روی results.items()، برای هر آزمایش دو خط رسم کنید – یکی برای train_acc (مثلاً "o" marker) و یکی برای test_acc (مثلاً "x" marker)، خط‌چین، هرکدام با label مشخص تا legend خوانا باشد.

برای تفسیر، از همین جدول فایل Session 2 استفاده کنید:

مشاهده	تفسیر ممکن
دقت train بالا، دقت test به طور محسوس پایین تر	overfitting یا ناهماهنگی train/test
دقت train و test هر دو پایین	underfitting، آموزش ناکافی، یا داده ی سخت
dropout دقت train را کم می کند اما شکاف را می بندد	ممکن است regularization به generalization کمک کرده باشد
augmentation هر دو امتیاز را کم می کند	سیاست augmentation شاید خیلی قوی، لیبل شکن، یا آموزش خیلی کوتاه است

سؤالات مفهومی C4

- طبق این جدول، کدام آزمایش شما بیشترین نشانه ی overfitting را دارد؟
- آیا dropout شکاف train/test را کم کرد؟ نتیجه ی augmentation چطور بود؟ به داده های واقعی خودتان (نه فرضی) اشاره کنید.

بلوک D – چالش یکپارچه ی پایانی ۲۵ دقیقه

D1 – دیباگ End-to-End و بازنویسی کامل از صفر ۱۵ دقیقه

کد زیر روی همان CIFAR-10 است و ۵ مشکل دارد – ترکیبی از الگوهای هر دو فایل:

```
final_model = RegularizedCNN(num_classes=len(class_names), dropout_p=0.3) # بدون .to(DEVICE)
final_optimizer = torch.optim.Adam(final_model.parameters(), lr=1e-3)

leaky_test_loader = DataLoader(
    Subset(train_augmented_full, range(TEST_LIMIT)), # «تصادفی برای «تست augmentation از
    # استفاده شده
    batch_size=64, shuffle=True,
)

for images, labels in train_loader:
    images = images.to(DEVICE)
    probs = torch.softmax(final_model(images), dim=1)
    loss = nn.CrossEntropyLoss()(probs, labels)
    loss.backward()
    final_optimizer.step()

predicted = probs # استفاده شده probs مستقیماً از argmax به جای #
```

فقط پیدا کردن کافی نیست: هر ۵ مشکل را نام ببرید (و بگویید به کدام درس از کدام فایل مربوط است)، سپس یک نسخه ی کاملاً بازنویسی شده و درست از کل این قطعه کد را از صفر بنویسید – با موتور آموزش صحیح بلوک B، یک test_loader واقعی (نه leaky)، و device handling درست.

D2 – تصمیم گیری یکپارچه و جمع بندی ۱۰ دقیقه

با تکیه بر همان نتایج واقعی خودتان از بلوک های B و C – نه فرضی:

1. کدام تنظیم baseline/dropout/augmentation را برای این مسئله پیشنهاد می‌کنید؟ به منحنی‌های واقعی خودتان ارجاع دهید.
2. اگر بخواهید دقت را بیشتر بهبود دهید ولی فقط با ابزارهایی که امروز یاد گرفتید، بدون batch normalization یا ResNet که هنوز تدریس نشده‌اند، چه یک گام بعدی منطقی پیشنهاد می‌دهید؟ مثال قابل قبول: افزایش TRAIN_LIMIT/epochs، امتحان "pooling="average"، تنظیم dropout_p.
3. یک جمله بنویسید: امروز کدام مفهوم بین سه فایل (Loss، موتور آموزش، Generalization) را با یک مثال واقعی از کد خودتان به هم وصل کردید؟